Tower Of Hanoi Program and Recursion Ananlysis –Recursion Tree

TOH(Tower of Hanoi) program as given below:

```
// Tower of Hanoi

#include <iostream>
using namespace std;

void toh(int n, char src, char dest, char aux)
{
    if (n == 0)
    {
        return ;
    }


    toh(n - 1, src, aux, dest);

    cout << "Move " << n << " from " << src << " to " <<
dest << endl;

    toh(n - 1, aux, dest, src);
}

int main()
{

    toh(3, 'A', 'C', 'B');
    return 0;
}
```

Hence we have 3 disk($n = 3$) and 3 towers where A is source (src), C is Destination($dest$) and B is auxiliary(aux).

Approach:-

1. ***First Shuffle:***

   From Source(src) tower disks will move to auxiliary(aux) tower using destination($dest$). And no. disk required to move is $n - 1$.

   To

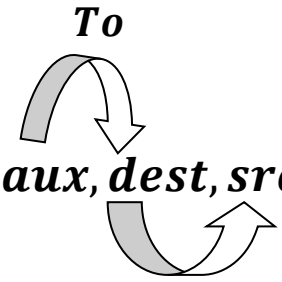
   $\therefore TOH(n-1, src, aux, dest);$

   using

2. ***Final Shuffle:***

   Again from auxiliary(aux) tower disks will move to destination($dest$) tower using source(src). And no. disk required to move is $n - 1$.

$$\therefore TOH(n-1, aux, dest, src);$$



Using these shuffles the above program will be formed.

Now let see working of $TOH(3,' A',' C',' B')$

Before going to the stack process . Lets see what happens here generally.

So 1st Initialization: $TOH(3,' A',' C',' B')$, ***where A is src, C is dest and B is aux .***

- ***Next, It will call*** $TOH(n-1, src, aux, dest)$ ***project as:*** $TOH(3-1=2,'A','B','C')$ **,**
  this shuffling of A, B, C ***makes initialization to*** $TOH(n-1, src, dest, aux)$ ***while calling*** $i.e.\ src = A, dest = B,$ ***and*** $aux = C.$

- $\boldsymbol{Next, It\ will\ again\ call\ TOH(n-1, src, aux, dest)\ project\ as: TOH(2-1=1, 'A', 'C', 'B')\ ,\ this\ shuffling\ of\ A, B, C\ makes\ initialization\ to\ TOH(n-1, src, dest, aux)\ while\ calling\ i.e.\ src = A, dest = C, and\ aux = B.}$

- $\boldsymbol{It\ then\ prints: Move\ 1\ from\ A\ to\ C.\ i.e.}$

```
cout << "Move " << n << " from " << src << " to " << dest << endl;
```

- $\boldsymbol{Next, It\ will\ again\ call\ TOH(n-1, src, aux, dest)\ hence\ , n-1\ will\ be\ 0\ , therefore\ executes\ the\ base\ case.}$

```
if (n == 0)
{
    return ;
}
```

- $\boldsymbol{Next, It\ backtracks\ to\ TOH(2, 'A', 'B', 'C')\ and\ src\ is\ A, dest\ is\ B\ , aux\ is\ C\ and\ n\ is\ 2\ according\ to\ TOH(n, src, dest, aux).}$

- $\boldsymbol{It\ then\ prints: Move\ 2\ from\ A\ to\ B.\ i.e.}$

```
cout << "Move " << n << " from " << src << " to " << dest << endl;
```

- $Next, It\ will\ call\ \ TOH(n-1, aux, dest, src)\ results:$ $TOH(2-1=1, 'C', 'B', 'A')\ and\ assigns$ $TOH(n, src, dest, aux)\ i.e.\ src = C, dest = B\ and$ $aux = A.$

- $It\ then\ prints: Move\ 1\ from\ C\ to\ B. i.e.$

```
cout << "Move " << n << " from " << src << " to " <<
dest << endl;
```

- $Next, It\ will\ again\ call\ TOH(n-1, aux, dest, src)$ $hence, n-1\ will\ be\ 0, therefore\ executes$ $the\ base\ case.$

- $Next, It\ backtracks\ to\ TOH(3, 'A', 'C', 'B')\ and$ $src\ is\ A, dest\ is\ C, aux\ is\ B\ and\ n\ is\ 2$ $according\ to\ TOH(n, src, dest, aux).$

- $It\ then\ prints: Move\ 3\ from\ A\ to\ C. i.e.$

```
cout << "Move " << n << " from " << src << " to " <<
dest << endl;
```

- $Next, It\ will\ call\ TOH(n-1, aux, dest, src)\ project\ as:$ $TOH(3-1=2, 'B', 'C', 'A')\ ,$ $this\ shuffling\ of\ A, B, C\ makes$ $initialization\ to\ TOH(n-1, src, dest, aux)\ while$ $calling\ i.e.\ src = B, dest = C, and\ aux = A.$

- $Next, It\ will\ call\ TOH(n-1, src, aux, dest)\ project\ as:$
  $TOH(2-1=1, 'B', 'A', 'C')$,
  $this\ shuffling\ of\ A, B, C\ makes$
  $initialization\ to\ TOH(n-1, src, dest, aux)\ while$
  $calling\ i.e.\ src = B, dest = A, and\ aux = C.$

- $It\ then\ prints: Move\ 1\ from\ B\ to\ A.\ i.e.$

```
cout << "Move " << n << " from " << src << " to " <<
dest << endl;
```

- $Next, It\ will\ again\ call\ TOH(n-1, src, aux, dest)$
  $hence\ , n-1\ will\ be\ 0\ , therefore\ executes$
  $the\ base\ case.$

```
if (n == 0)
{
    return ;
}
```

- $Next, It\ backtracks\ to\ TOH(2, 'B', 'C', 'A')\ and$
  $src\ is\ B, dest\ is\ C\ , aux\ is\ A\ and\ n\ is\ 2$
  $according\ to\ TOH(n, src, dest, aux).$

- $It\ then\ prints: Move\ 2\ from\ B\ to\ C.\ i.e.$

```
cout << "Move " << n << " from " << src << " to " <<
dest << endl;
```

- ***Next, It will call*** $TOH(n-1, aux, dest, src)$ ***results***: $TOH(2-1=1, 'A', 'C', 'B')$ ***and assigns*** $TOH(n, src, dest, aux)$ $i.e.\ src = A, dest = C$ ***and*** $aux = B$.

- ***It then prints*** : ***Move*** 1 $from\ A\ to\ C.\ i.e.$

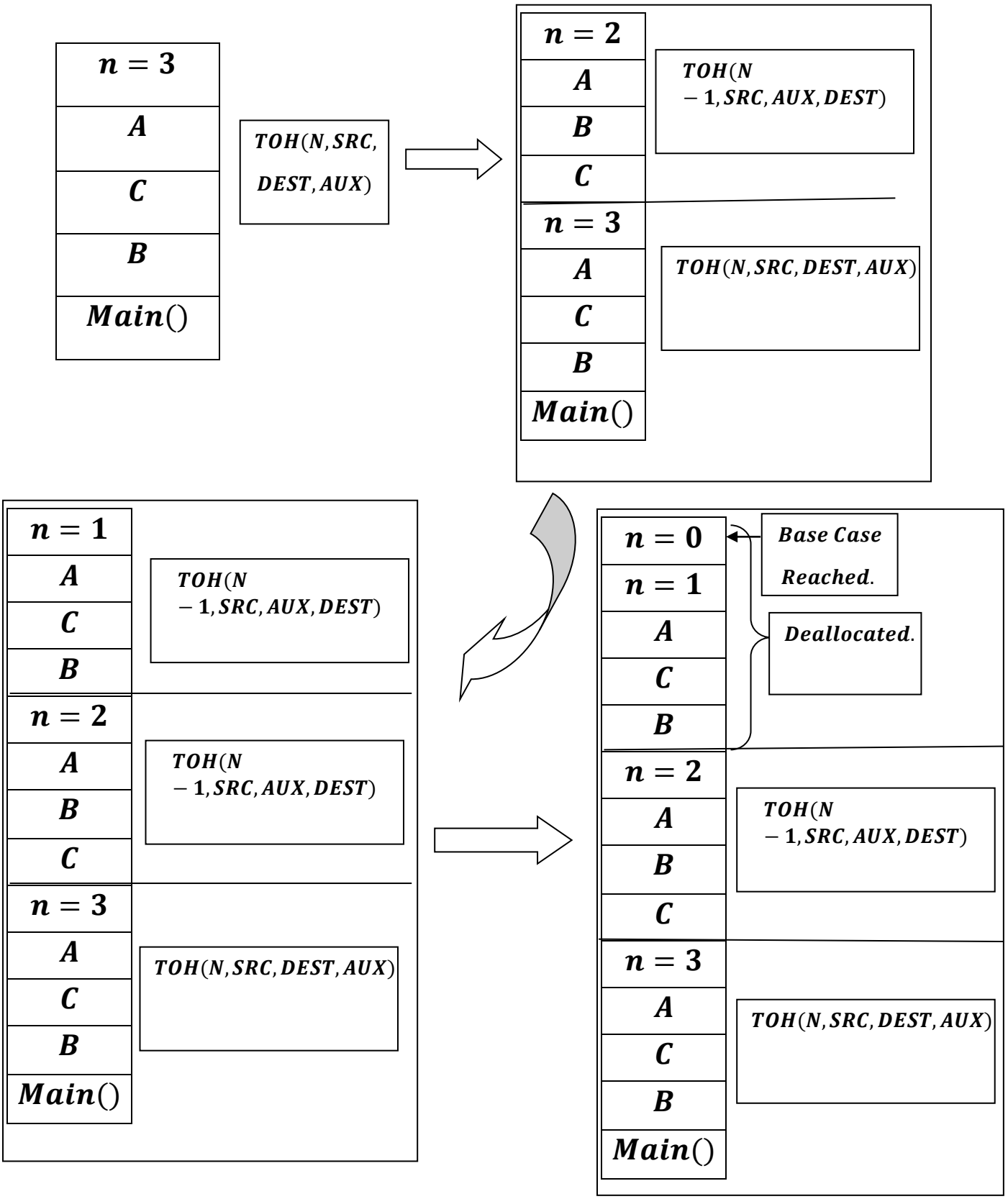
```
cout << "Move " << n << " from " << src << " to " <<
dest << endl;
```

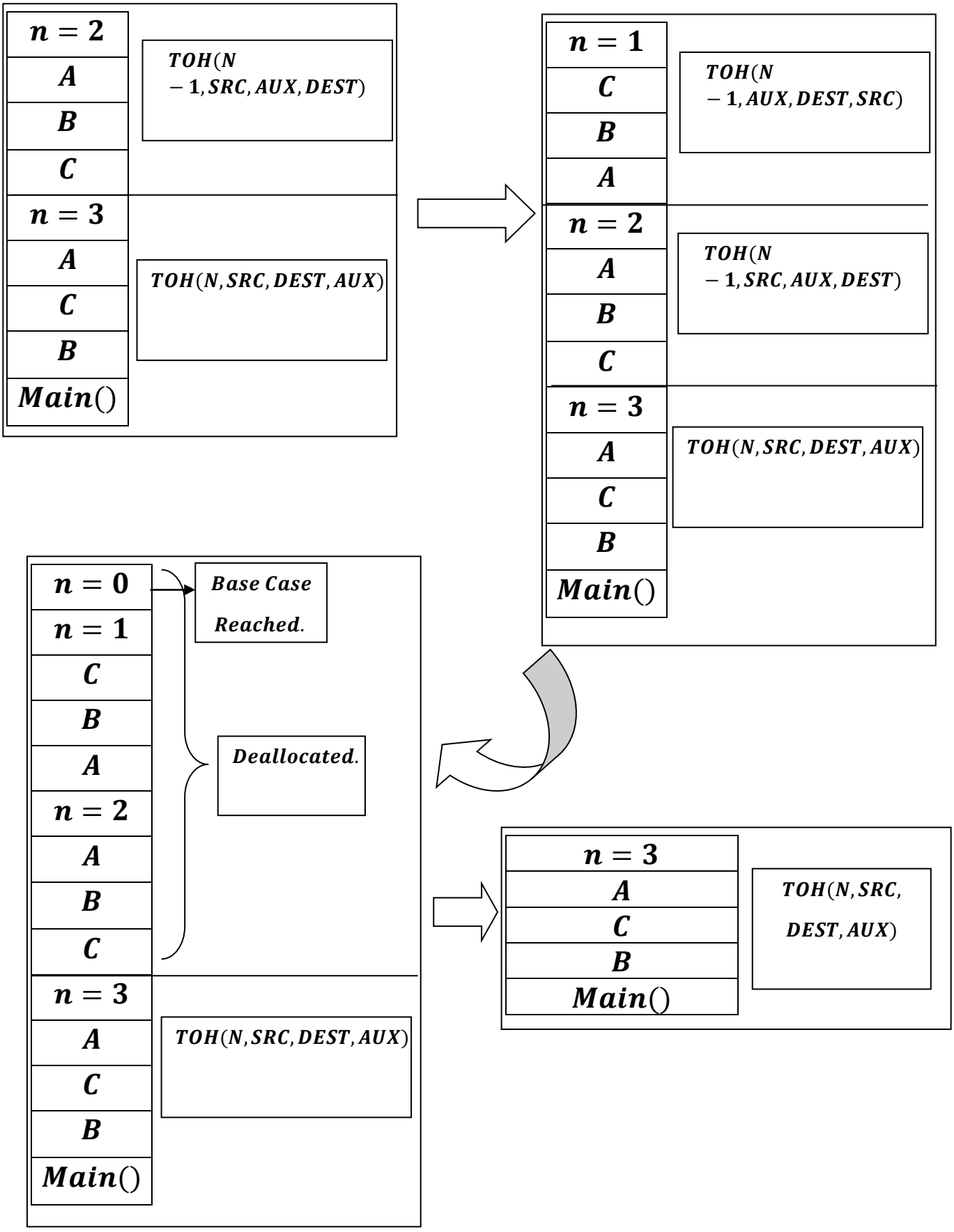
- ***Next, It will again call*** $TOH(n-1, aux, dest, src)$ ***hence*** , $n-1$ ***will be*** 0 , $therefore\ executes$ ***the base case***.

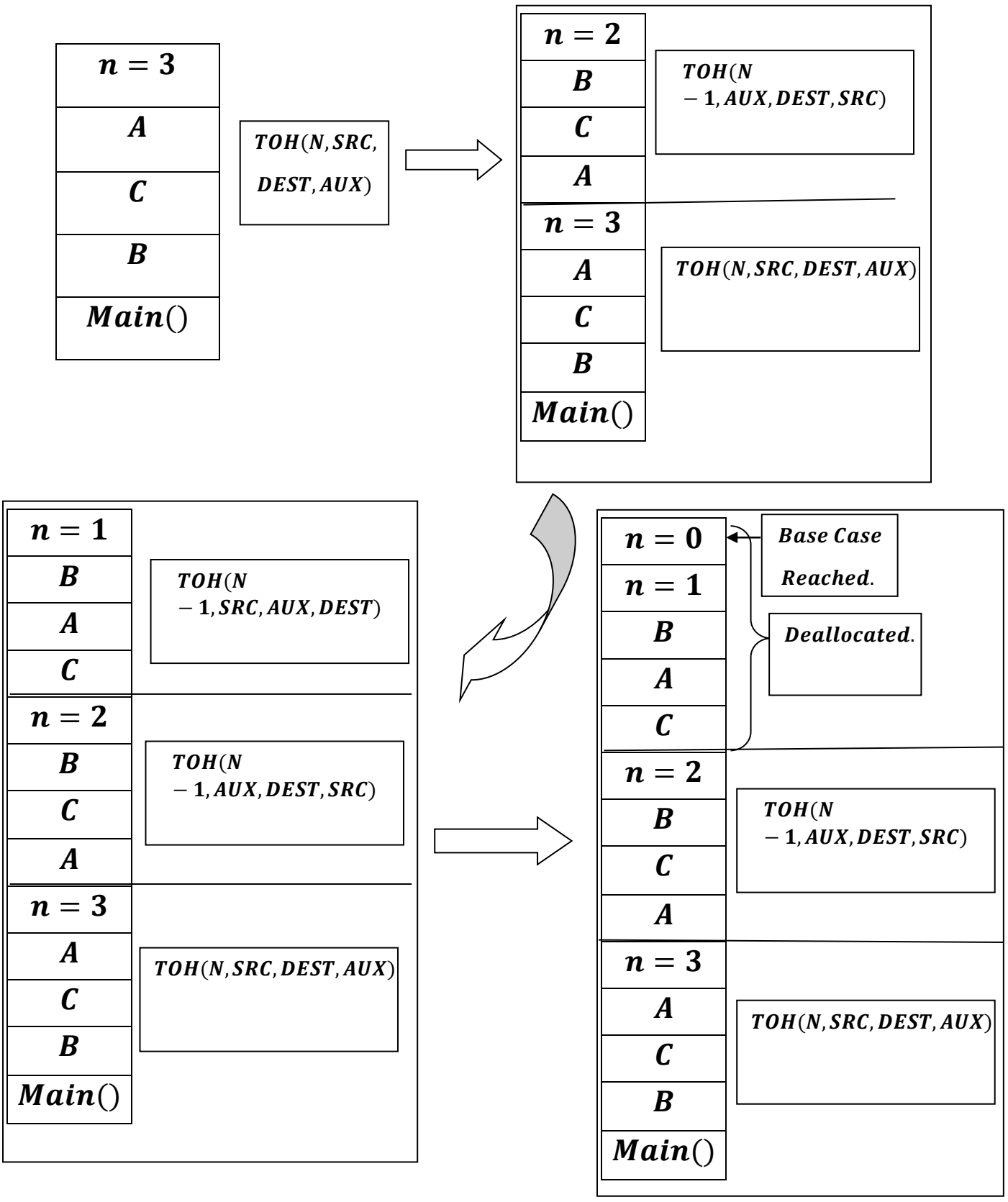
```
if (n == 0)
{
    return ;
}
```

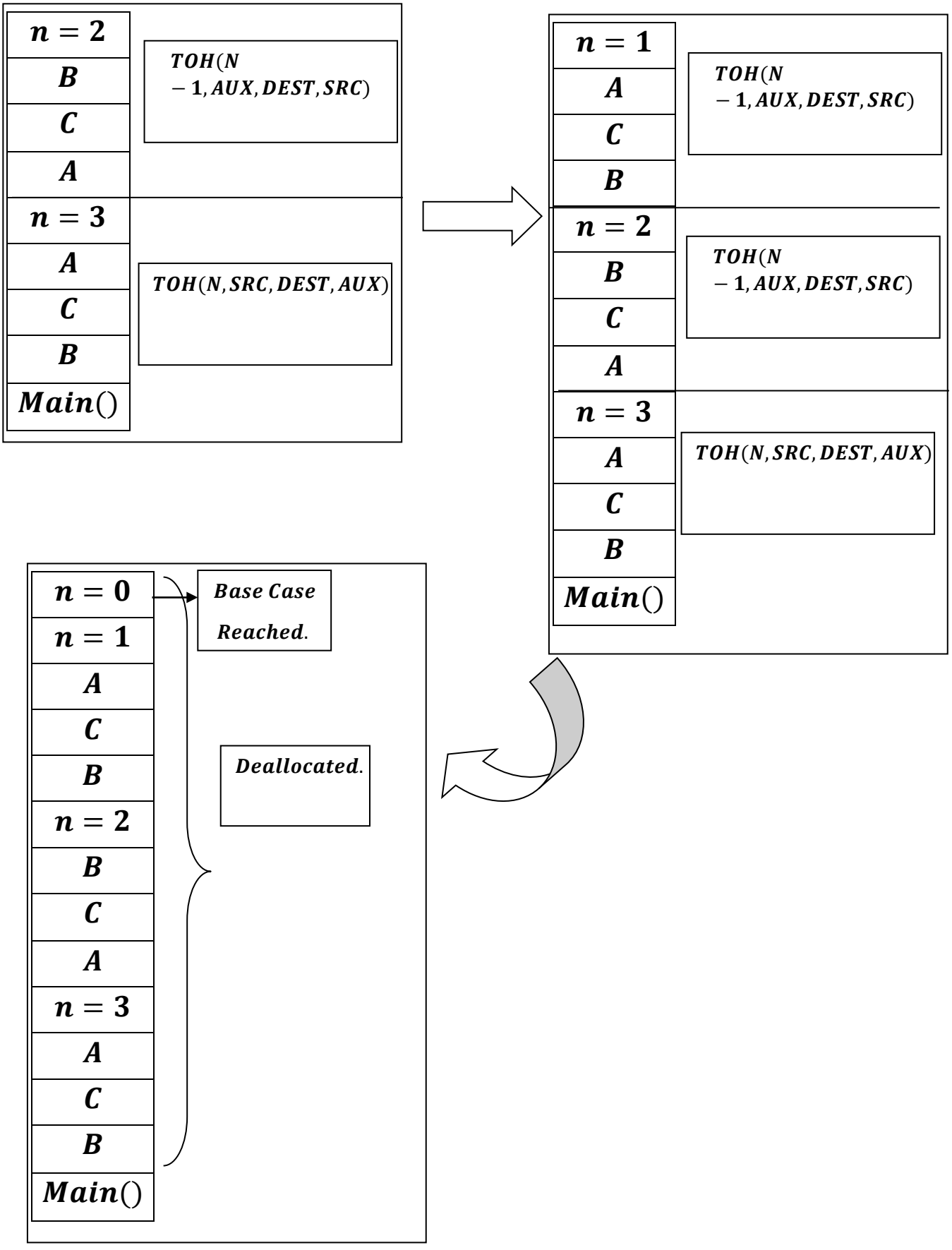
Hence it ends here.

Now let see working of Stack



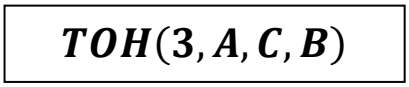

n = 2
A
B
C
TOH(N − 1, SRC, AUX, DEST)
n = 3
A
C
B
Main()
TOH(N, SRC, DEST, AUX)
n = 1
C
B
A
TOH(N − 1, AUX, DEST, SRC)
n = 2
A
B
C
TOH(N − 1, SRC, AUX, DEST)
n = 3
A
C
B
Main()
TOH(N, SRC, DEST, AUX)
n = 0
n = 1
C
B
A
n = 2
A
B
C
Base Case Reached.
Deallocated.
n = 3
A
C
B
Main()
TOH(N, SRC, DEST, AUX)
n = 3
A
C
B
Main()
TOH(N, SRC, DEST, AUX)


n = 3
A
C
B
Main()
TOH(N, SRC, DEST, AUX)
n = 2
B
C
A
TOH(N − 1, AUX, DEST, SRC)
n = 3
A
C
B
Main()
TOH(N, SRC, DEST, AUX)
n = 1
B
A
C
TOH(N − 1, SRC, AUX, DEST)
n = 2
B
C
A
TOH(N − 1, AUX, DEST, SRC)
n = 3
A
C
B
Main()
TOH(N, SRC, DEST, AUX)
n = 0
Base Case Reached.
n = 1
B
A
C
Deallocated.
n = 2
B
C
A
TOH(N − 1, AUX, DEST, SRC)
n = 3
A
C
B
Main()
TOH(N, SRC, DEST, AUX)

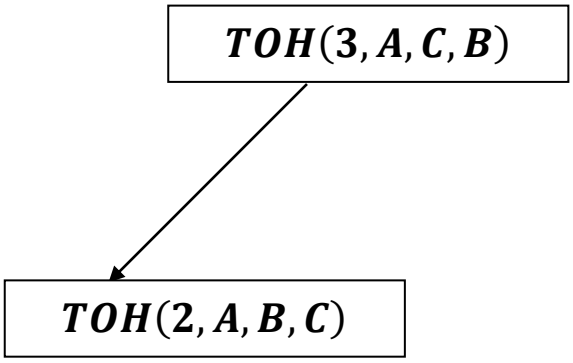

n = 2
B
C
A
TOH(N − 1, AUX, DEST, SRC)
n = 3
A
C
B
Main()
TOH(N, SRC, DEST, AUX)
n = 1
A
C
B
TOH(N − 1, AUX, DEST, SRC)
n = 2
B
C
A
TOH(N − 1, AUX, DEST, SRC)
n = 3
A
C
B
Main()
TOH(N, SRC, DEST, AUX)
n = 0
Base Case Reached.
n = 1
A
C
B
n = 2
B
C
A
n = 3
A
C
B
Main()
Deallocated.

$Creation\ of\ Recursion\ Tree\ for$
$TOH(3, src =' A' , dest =' C' \ and \ aux =' B')$

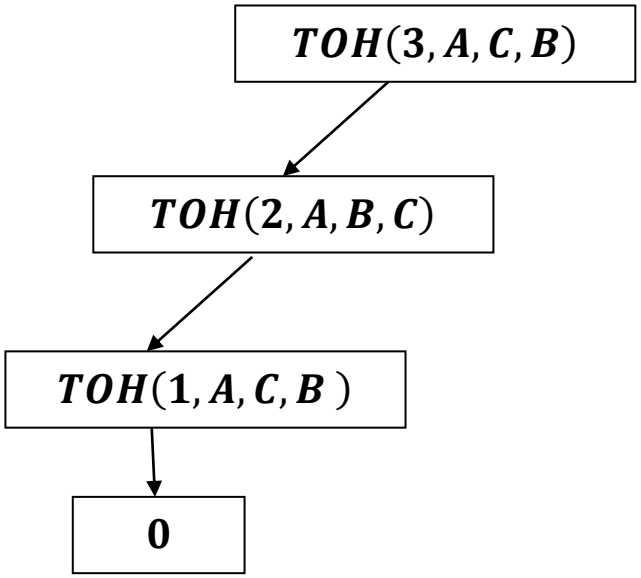
1. $Initialization.\ (1st\ step\ )$



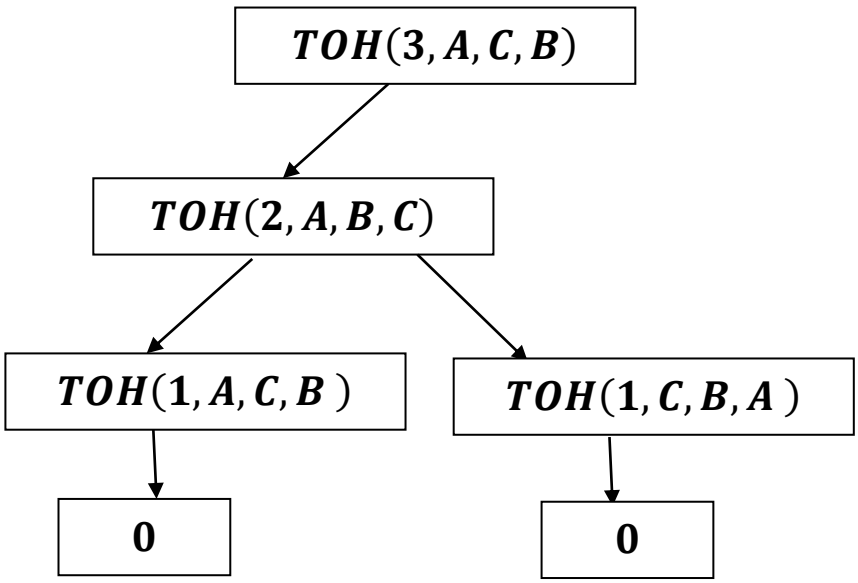
2. $2nd\ Step$



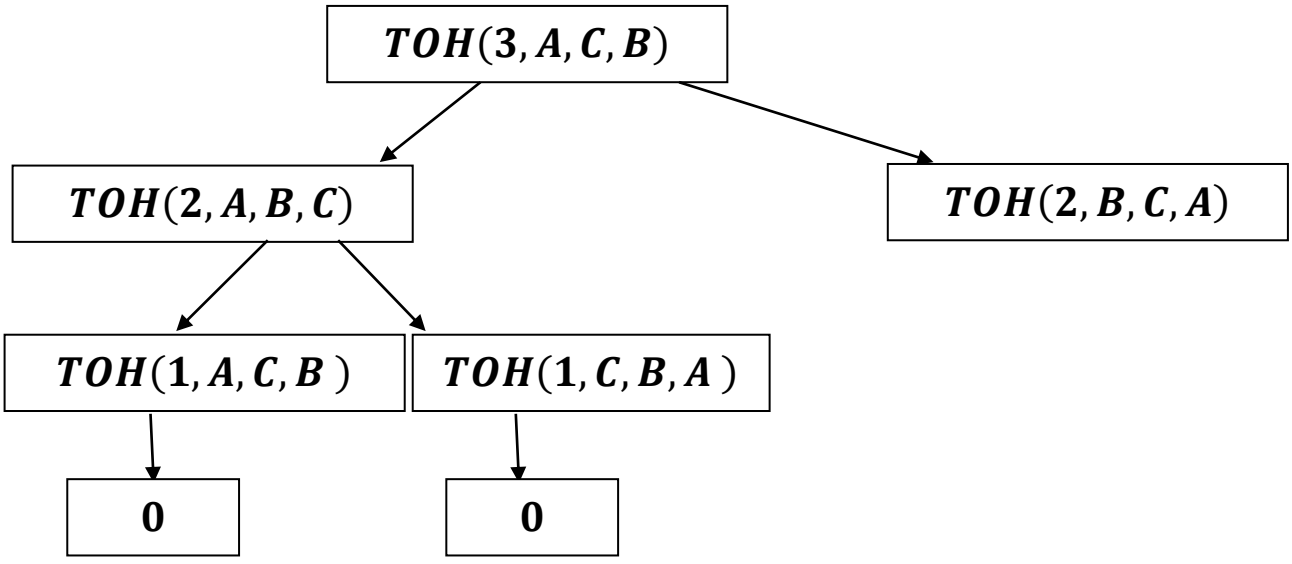
3. $3rd\ Step$



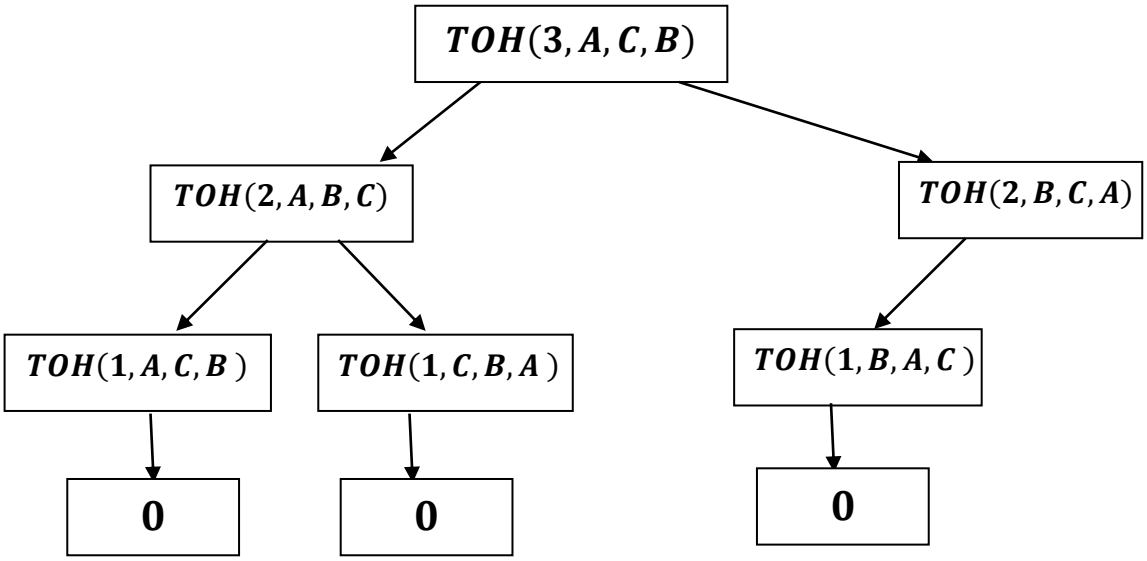
4. 4*rth Step*



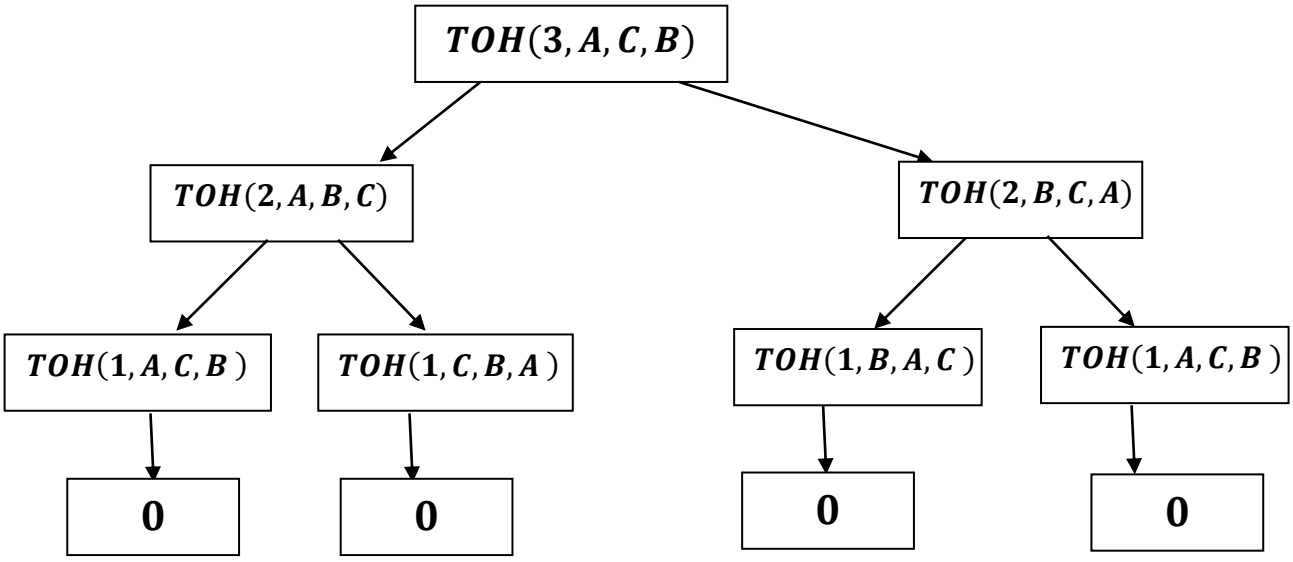
5. 5*th Step*



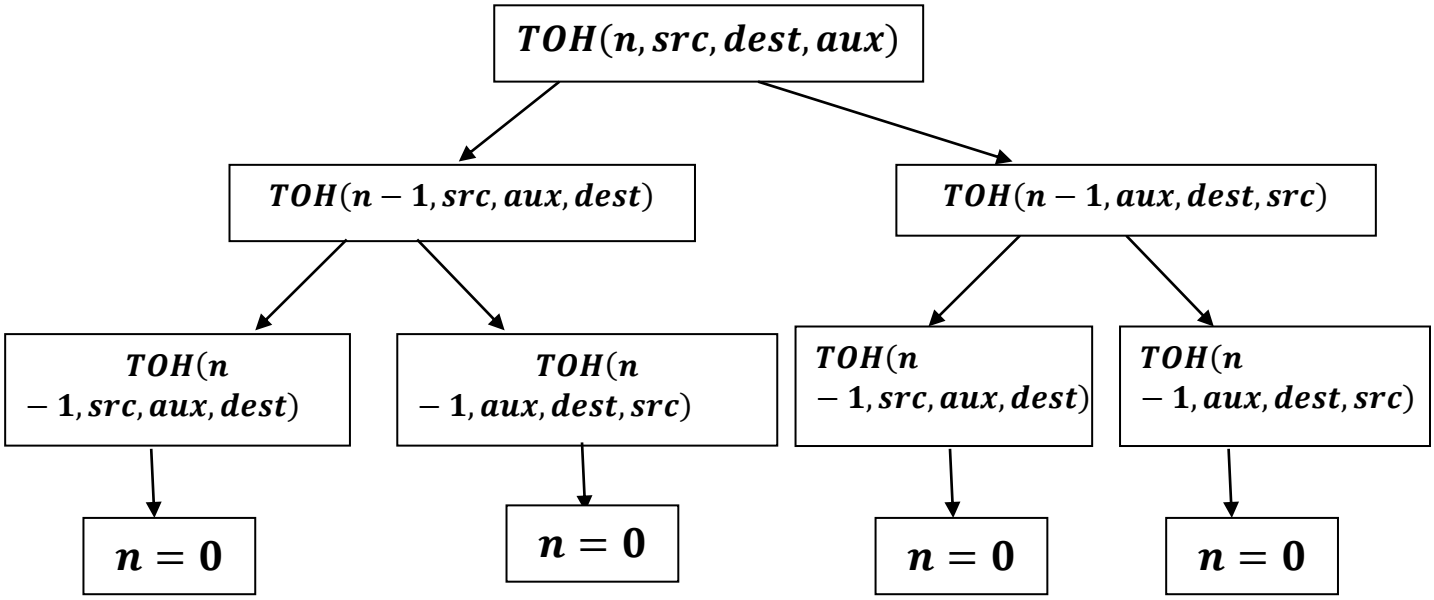
6. 6*th Step*



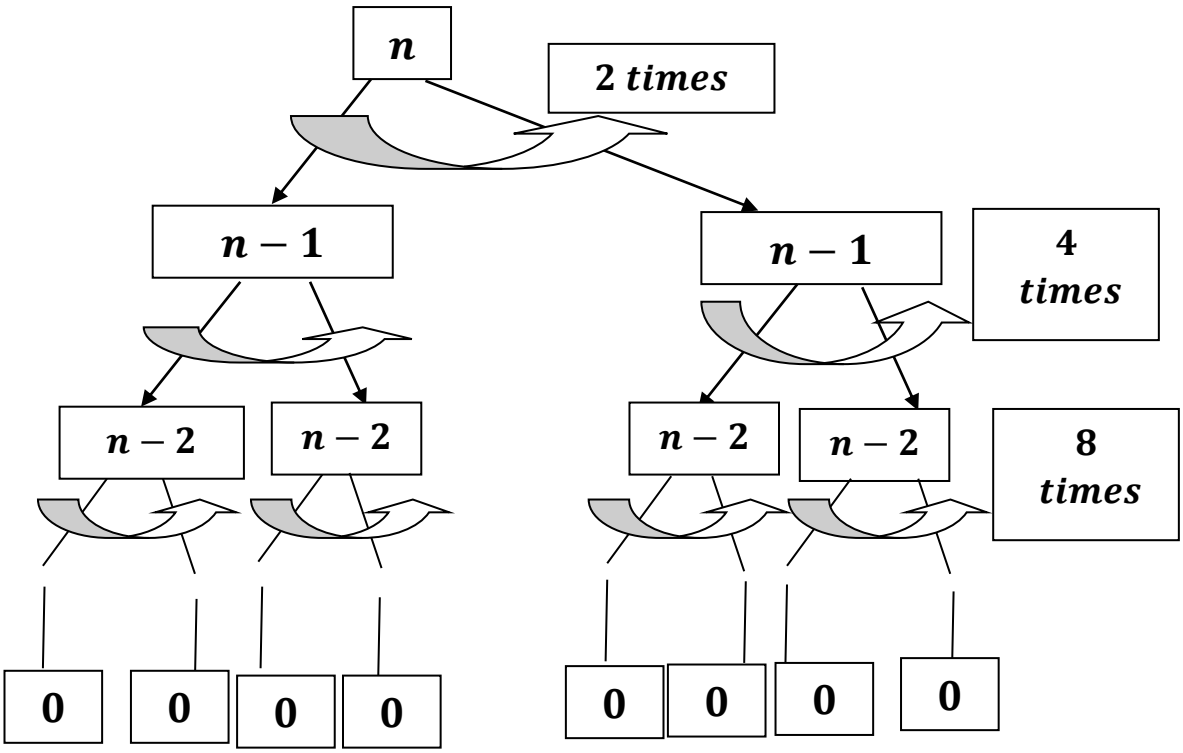
7. 7*th Step*



***We can also arrange the recurrence relation as*:**



***If the recursion takes place `n` times then the recurrence tree occurs*:**



****